

MAKALAH
“TUGAS PRAKTIKUM 7 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Cynthia Citra Azaria
NIM:2410651025
Prodi:Teknik Informatika

UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026

Tugas 1: Analisis Memory Layout

Deskripsi

Modifikasi program `address_demo.c` untuk menganalisis memory layout dengan lebih detail.

Yang Harus Dikerjakan

A. Modifikasi Program

Tambahkan fitur-fitur berikut pada program:

- . Array Allocation: Buat array statis dan dinamis, tampilkan alamatnya
- . Multiple Functions: Buat 3 fungsi berbeda, tampilkan alamat setiap fungsi
- . Recursive Function: Buat fungsi rekursif dan tampilkan alamat stack di setiap level
- . String Literals: Tampilkan alamat string literals

```
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano address_demo.c
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ gcc -o address_demo address_demo.c
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ ./address_demo
=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
main() address : 0x57ebafe0d24a
func1() address : 0x57ebafe0d1c9
func2() address : 0x57ebafe0d1d4

2. Data Segment:
static_array[0] address : 0x57ebafe10010
string_literal address : 0x57ebafe0e008

3. Heap:
dynamic_array[0] address : 0x57ebc11796b0
malloc block 1 : 0x57ebc11796d0
malloc block 2 : 0x57ebc11796f0

4. Stack (Recursive Calls):
Level 0 local_var: 0x7ffe431d52d0
Level 1 local_var: 0x7ffe431d52a0
Level 2 local_var: 0x7ffe431d5274
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* ===== DATA SEGMENT ===== */
static int static_array[3] = {1,2,3};
const char *string_literal = "Hello Memory";

/* ===== FUNCTION DECLARATION ===== */
void func1();
void func2();
void recursive_func(int level);

/* ===== FUNCTIONS (TEXT SEGMENT) ===== */
void func1() {}
void func2() {}

/* ===== RECURSIVE FUNCTION ===== */
void recursive_func(int level) {
    int local_var = level;

    printf("Level %d local_var: %p\n", level, (void*)&local_var);

    if (level < 2) {
        recursive_func(level + 1);
    }
}

int main() {
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");

    /* ===== TEXT SEGMENT ===== */
    printf("1. Text Segment:\n");
    printf("main() address : %p\n", (void*)main);
    printf("func1() address : %p\n", (void*)func1);
    printf("func2() address : %p\n", (void*)func2);

    /* ===== DATA SEGMENT ===== */
    printf("2. Data Segment:\n");
    printf("static_array[0] address : %p\n", (void*)&static_array[0]);
    printf("string_literal address : %p\n", (void*)&string_literal);

    /* ===== HEAP ===== */
    printf("3. Heap:\n");
    int *dynamic_array = malloc(3 * sizeof(int));
    int *block1 = malloc(10);
    int *block2 = malloc(10);

    printf("dynamic_array[0] address : %p\n", (void*)&dynamic_array[0]);
    printf("malloc block 1 : %p\n", (void*)&block1);
    printf("malloc block 2 : %p\n", (void*)&block2);

    /* ===== STACK (RECURSIVE) ===== */
    printf("4. Stack (Recursive Calls):\n");
    recursive_func(0);

    free(dynamic_array);
    free(block1);
    free(block2);

    return 0;
}
```

B. Analisis

- Jarak Stack dan Heap
Stack berada di alamat tinggi dan heap di bawahnya, sehingga jaraknya cukup jauh untuk mencegah tabrakan memori.

- Jarak Text dan Data Segment
Text segment berada di alamat lebih rendah dan data segment tepat setelahnya, sehingga jaraknya relatif dekat.
- Jarak Alamat Fungsi
Alamat antar fungsi berdekatan karena berada pada text segment yang sama.
- Perubahan Alamat (5 kali eksekusi)
Alamat memori berubah setiap program dijalankan, kecuali pola urutannya.
- Penyebab (ASLR)
Hal ini disebabkan ASLR yang mengacak alamat memori untuk meningkatkan keamanan.
- Perbandingan dengan `/proc/[pid]/maps`
Alamat sesuai dengan memory map. Permission: text (r-x), data (rw-), heap (rw-), stack (rw-).

Tugas 2: Implementasi Memory Allocator

Deskripsi

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa.

Yang Harus Dikerjakan

1. Implementasi Next-Fit Algorithm
2. Tambahkan algoritma Next-Fit pada program `memory_allocator.c`:
3. Next-Fit: Seperti First-Fit, tapi mulai pencarian dari posisi alokasi terakhir
4. Fitur Fragmentasi
5. Tambahkan fungsi untuk menghitung fragmentasi:
6. Benchmark Testing
7. Buat test case untuk membandingkan 4 algoritma:

Outputnya:

```
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano memory_allocator.c
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ gcc memory_allocator.c -o memory_allocator
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ ./memory_allocator
ALGORITMA | WAKTU | FRAG | GAGAL
-----|-----|-----|-----
First-Fit | 22.00 µs | 33.33 % | 2
Best-Fit  | 8.00 µs  | 100.00 % | 1
Worst-Fit | 6.00 µs  | 0.00 %   | 0
Next-Fit  | 3.00 µs  | 20.00 % | 0
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ |
```

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MEMORY_SIZE 100
#define MAX_PROCESS 20

int memory[MEMORY_SIZE];
int last_position = 0;

/* ===== UTILITY ===== */
void reset_memory() {
    for (int i = 0; i < MEMORY_SIZE; i++)
        memory[i] = 0;
    last_position = 0;
}

void print_memory() {
    for (int i = 0; i < MEMORY_SIZE; i++)
        printf("%c", memory[i] == 0 ? '.' : '#');
    printf("\n");
}

/* ===== FRAGMENTATION ===== */
float hitung_fragmentasi() {
    int free_blocks = 0, total_free = 0;

    for (int i = 0; i < MEMORY_SIZE; i++) {
        if (memory[i] == 0) {
            total_free++;
            if (i == 0 || memory[i - 1] != 0)
                free_blocks++;
        }
    }
    if (total_free == 0) return 0;
    return (free_blocks * 100.0) / total_free;
}

/* ===== ALGORITHMS ===== */
int first_fit(int pid, int size) {
    int count = 0;
    for (int i = 0; i < MEMORY_SIZE; i++) {
        if (memory[i] == 0) count++;
        else count = 0;

        if (count == size) {
            for (int j = i - size + 1; j <= i; j++)
                memory[j] = pid;
            return i - size + 1;
        }
    }
    return -1;
}

int best_fit(int pid, int size) {
    int best_start = -1, best_size = MEMORY_SIZE + 1;
    int i = 0;

    while (i < MEMORY_SIZE) {

```

```

        if (memory[i] == 0) {
            int start = i;
            int len = 0;
            while (i < MEMORY_SIZE && memory[i] == 0) {
                len++; i++;
            }
            if (len >= size && len < best_size) {
                best_size = len;
                best_start = start;
            }
        } else i++;
    }

    if (best_start != -1) {
        for (int j = best_start; j < best_start + size; j++)
            memory[j] = pid;
    }
    return best_start;
}

int worst_fit(int pid, int size) {
    int worst_start = -1, worst_size = 0;
    int i = 0;

    while (i < MEMORY_SIZE) {
        if (memory[i] == 0) {
            int start = i;
            int len = 0;
            while (i < MEMORY_SIZE && memory[i] == 0) {
                len++; i++;
            }
            if (len >= size && len > worst_size) {
                worst_size = len;
                worst_start = start;
            }
        } else i++;
    }

    if (worst_start != -1) {
        for (int j = worst_start; j < worst_start + size; j++)
            memory[j] = pid;
    }
    return worst_start;
}

int next_fit(int pid, int size) {
    int count = 0, start = last_position;

    for (int i = start; i < MEMORY_SIZE; i++) {
        if (memory[i] == 0) count++;
        else count = 0;

        if (count == size) {
            for (int j = i - size + 1; j <= i; j++)
                memory[j] = pid;
            last_position = i + 1;
            return i - size + 1;
        }
    }
}

```

```

        count = 0;
        for (int i = 0; i < start; i++) {
            if (memory[i] == 0) count++;
            else count = 0;

            if (count == size) {
                for (int j = i - size + 1; j <= i; j++)
                    memory[j] = pid;
                last_position = i + 1;
                return i - size + 1;
            }
        }
        return -1;
    }

/* ===== BENCHMARK ===== */
void benchmark(const char *name, int (*alloc)(int,int)) {
    reset_memory();
    int failed = 0;
    clock_t start = clock();

    for (int i = 1; i <= MAX_PROCESS; i++) {
        int size = rand() % 10 + 1;
        if (alloc(i, size) == -1)
            failed++;
    }

    clock_t end = clock();
    double waktu = (double)(end - start) / CLOCKS_PER_SEC * 1000000;

    printf("%-10s | %8.2f µs | %6.2f %% | %d\n",
           name, waktu, hitung_fragmentasi(), failed);
}

/* ===== MAIN ===== */
int main() {
    srand(time(NULL));

    printf("ALGORITMA | WAKTU | FRAG | GAGAL\n");
    printf("-----\n");

    benchmark("First-Fit", first_fit);
    benchmark("Best-Fit", best_fit);
    benchmark("Worst-Fit", worst_fit);
    benchmark("Next-Fit", next_fit);

    return 0;
}

```

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	22.00	33.33 %	2
Best-Fit	8.00	100.00 %	1
Worst-Fit	6.00	0.00 %	0
Next-Fit	3.00	20.00 %	0

Berdasarkan hasil pengujian, Next-Fit merupakan algoritma tercepat, sedangkan Worst-Fit paling efektif dalam mengurangi fragmentasi. Pemilihan algoritma sebaiknya disesuaikan dengan kebutuhan sistem, apakah mengutamakan kecepatan atau efisiensi memori.

Tugas 3: Advanced Paging System

Deskripsi

Kembangkan sistem paging dengan page replacement algorithm dan statistik.

Yang Harus Dikerjakan

A. Implementasi Page Replacement - FIFO

Modifikasi paging_simulator.c untuk menangani situasi ketika semua frame penuh:

B. Implementasi Page Replacement - LRU Tambahkan algoritma LRU (Least Recently Used):

C. Statistik dan Analisis

D. Tambahkan fungsi untuk menghitung:

```
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano memory_allocator.c
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano paging_simulator.c
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ gcc paging_simulator.c -o paging_simulator
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ ./paging_simulator

=== FIFO ===
Ref 7: 7 -1 -1 -1
Ref 0: 7 0 -1 -1
Ref 1: 7 0 1 -1
Ref 2: 7 0 1 2
Ref 0: 7 0 1 2
Ref 3: 3 0 1 2
Ref 0: 3 0 1 2
Ref 4: 3 4 1 2
Ref 2: 3 4 1 2
Ref 3: 3 4 1 2
Ref 0: 3 4 0 2
Ref 3: 3 4 0 2
Ref 2: 3 4 0 2
Page Fault: 7
Hit Ratio : 0.46
Disk Write: 0

=== LRU ===
Ref 7: 7 -1 -1 -1
Ref 0: 7 0 -1 -1
Ref 1: 7 0 1 -1
Ref 2: 7 0 1 2
Ref 0: 7 0 1 2
Ref 3: 3 0 1 2
Ref 0: 3 0 1 2
Ref 4: 3 0 4 2
Ref 2: 3 0 4 2
Ref 3: 3 0 4 2
Ref 0: 3 0 4 2
Ref 3: 3 0 4 2
Ref 2: 3 0 4 2
Page Fault: 6
Hit Ratio : 0.54
Disk Write: 0
sintia@LAPTOP-PPED66E9:~/praktikum-memory/architecture$
```

```

#include <stdio.h>
#include <stdbool.h>

#define MAX_FRAMES 5
#define MAX_REF 50

int frames[MAX_FRAMES];
int last_used[MAX_FRAMES];
bool dirty[MAX_FRAMES];

int time_counter = 0;
int page_fault = 0;
int page_hit = 0;
int disk_write = 0;

/* cek page ada di frame atau tidak */
int find_page(int page, int nframes) {
    for (int i = 0; i < nframes; i++) {
        if (frames[i] == page)
            return i;
    }
    return -1;
}

/* FIFO replacement */
void simulate_fifo(int ref[], int nref, int nframes) {
    printf("\n=== FIFO ===\n");
    for (int i = 0; i < nframes; i++)
        frames[i] = -1;

    int fifo_index = 0;
    page_fault = page_hit = disk_write = 0;

    for (int i = 0; i < nref; i++) {
        int page = ref[i];
        int idx = find_page(page, nframes);

        if (idx != -1) {
            page_hit++;
        } else {
            page_fault++;
            if (frames[fifo_index] != -1 && dirty[fifo_index])
                disk_write++;

            frames[fifo_index] = page;
            dirty[fifo_index] = false;
            fifo_index = (fifo_index + 1) % nframes;
        }

        printf("Ref %d: ", page);
        for (int j = 0; j < nframes; j++)
            printf("%2d ", frames[j]);
        printf("\n");
    }

    printf("Page Fault: %d\n", page_fault);
    printf("Hit Ratio : %.2f\n", (float)page_hit / nref);
    printf("Disk Write: %d\n", disk_write);
}

/* LRU replacement */
void simulate_lru(int ref[], int nref, int nframes) {
    printf("\n=== LRU ===\n");
    for (int i = 0; i < nframes; i++) {
        frames[i] = -1;
        last_used[i] = 0;
    }

    page_fault = page_hit = disk_write = 0;
    time_counter = 0;

    for (int i = 0; i < nref; i++) {
        int page = ref[i];
        time_counter++;
        int idx = find_page(page, nframes);

        if (idx != -1) {
            page_hit++;
            last_used[idx] = time_counter;
        } else {
            page_fault++;

            int lru = 0;
            for (int j = 1; j < nframes; j++)
                if (last_used[j] < last_used[lru])
                    lru = j;

            if (frames[lru] != -1 && dirty[lru])
                disk_write++;

            frames[lru] = page;
            last_used[lru] = time_counter;
            dirty[lru] = false;
        }

        printf("Ref %d: ", page);
        for (int j = 0; j < nframes; j++)
            printf("%2d ", frames[j]);
        printf("\n");
    }

    printf("Page Fault: %d\n", page_fault);
    printf("Hit Ratio : %.2f\n", (float)page_hit / nref);
    printf("Disk Write: %d\n", disk_write);
}

int main() {
    int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2};
    int nref = 13;
    int frames_count = 4;

    simulate_fifo(reference_string, nref, frames_count);
    simulate_lru(reference_string, nref, frames_count);

    return 0;
}

```

Tabel trace-FIFO

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Ya
2	0	7	0	-	-	Ya
3	1	7	0	1	-	Ya
4	2	7	0	1	2	Ya
5	0	7	0	1	2	Tidak
6	3	3	0	1	2	Ya
7	0	3	0	1	2	Tidak
8	4	3	4	1	2	Ya
9	2	3	4	1	2	Tidak
10	3	3	4	1	2	Tidak
11	0	3	4	0	2	Ya
12	3	3	4	0	2	Tidak
13	2	3	4	0	2	Tidak

tabel trace LRU

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Ya
2	0	7	0	-	-	Ya
3	1	7	0	1	-	Ya
4	2	7	0	1	2	Ya
5	0	7	0	1	2	Tidak
6	3	3	0	1	2	Ya
7	0	3	0	1	2	Tidak
8	4	3	0	4	2	Ya
9	2	3	0	4	2	Tidak
10	3	3	0	4	2	Tidak
11	0	3	0	4	2	Tidak
12	3	3	0	4	2	Tidak
13	2	3	0	4	2	Tidak

Belady's Anomaly (Analisis)

- FIFO:
 - Page fault tidak selalu berkurang saat frame ditambah
 - Bisa terjadi Belady's Anomaly
- LRU:
 - Page fault menurun atau tetap
 - Tidak mengalami Belady's Anomaly

LRU lebih efisien, FIFO lebih simpel dan murah secara implementasi.

Tugas 4: Multi-Level Page Table

Deskripsi

Implementasikan dan bandingkan berbagai struktur page table.

Yang Harus Dikerjakan

- Three-Level Page Table
- Kembangkan three-level page table system:
- Implementasikan inverted page table:
- Analisis Perbandingan (Dalam Laporan)

```
#include <stdio.h>
#include <stdlib.h>

#define L1 4
#define L2 4
#define L3 4
#define FRAMES 8

/* =====
THREE-LEVEL PAGE TABLE
===== */

typedef struct {
    int frame;
    int valid;
} PageEntry;

typedef struct {
    PageEntry entry[L3];
} PT_Level3;

typedef struct {
    PT_Level3* table[L2];
} PT_Level2;

typedef struct {
    PT_Level2* table[L1];
} PT_Level1;

PT_Level1* three_level_pt;

void init_three_level() {
    three_level_pt = malloc(sizeof(PT_Level1));
    for (int i = 0; i < L1; i++)
        three_level_pt->table[i] = NULL;
}

void map_three_level(int a, int b, int c, int frame) {
    if (!three_level_pt->table[a]) {
        three_level_pt->table[a] = malloc(sizeof(PT_Level2));
        for (int i = 0; i < L2; i++)
            three_level_pt->table[a]->table[i] = NULL;
    }

    if (!three_level_pt->table[a]->table[b]) {
        three_level_pt->table[a]->table[b] = malloc(sizeof(PT_Level3));
        for (int i = 0; i < L3; i++)
            three_level_pt->table[a]->table[b]->entry[i].valid = 0;
    }

    three_level_pt->table[a]->table[b]->entry[c].frame = frame;
    three_level_pt->table[a]->table[b]->entry[c].valid = 1;
}

void translate_three_level(int a, int b, int c) {
    printf("[Three-Level] ");
    if (three_level_pt->table[a] &&
        three_level_pt->table[a]->table[b] &&
        three_level_pt->table[a]->table[b]->entry[c].valid) {
        printf("Frame = %d\n",
            three_level_pt->table[a]->table[b]->entry[c].frame);
    } else {
        printf("Page Fault!\n");
    }
}

/* =====
INVERTED PAGE TABLE
===== */

typedef struct {
    int pid;
    int page;
    int valid;
} InvertedEntry;

InvertedEntry inverted_pt[FRAMES];

void init_inverted() {
    for (int i = 0; i < FRAMES; i++)
        inverted_pt[i].valid = 0;
}

void map_inverted(int pid, int page, int frame) {
    inverted_pt[frame].pid = pid;
    inverted_pt[frame].page = page;
    inverted_pt[frame].valid = 1;
}

void translate_inverted(int pid, int page) {
    printf("[Inverted] ");
    for (int i = 0; i < FRAMES; i++) {
        if (inverted_pt[i].valid &&
            inverted_pt[i].pid == pid &&
            inverted_pt[i].page == page) {
            printf("Frame = %d\n", i);
            return;
        }
    }
    printf("Page Fault!\n");
}

/* =====
MAIN PROGRAM
===== */

int main() {
    printf("=== MULTI-LEVEL PAGE TABLE DEMO ===\n\n");

    /* Three-Level Page Table Demo */
    init_three_level();
    map_three_level(1, 2, 3, 10);
    translate_three_level(1, 2, 3);
    translate_three_level(0, 1, 1);

    printf("\n");

    /* Inverted Page Table Demo */
```

```
/* Inverted Page Table Demo */
init_inverted();
map_inverted(1, 5, 4);
translate_inverted(1, 5);
translate_inverted(2, 1);

return 0;
}
```


Outputnya:

```
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano multilevel_paging.c
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ gcc multilevel_paging.c -o multilevel_paging
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ ./multilevel_paging
=== MULTI-LEVEL PAGE TABLE DEMO ===

[Three-Level] Frame = 10
[Three-Level] Page Fault!

[Inverted] Frame = 4
[Inverted] Page Fault!
```

Struktur	Entries	Size per Entry	Total Memory	Efisiensi dibanding Single-Level
Two-Level	Banyak (bergantung proses)	Sedang	Sedang	Lebih efisien
Three-Level	Lebih sedikit (dialokasikan bertahap)	Sedang	Lebih kecil	Lebih efisien lagi
Inverted	= jumlah frame fisik	Kecil	Paling kecil	Paling efisien

Kesimpulan:

Inverted Page Table memiliki memory overhead paling kecil karena jumlah entry tidak bergantung pada ukuran virtual address space.

2. Translation Speed

- Two-Level Page Table: $\pm 2-3$ kali akses memori
- Three-Level Page Table: $\pm 3-4$ kali akses memori
- Inverted Page Table: Bisa lebih banyak karena perlu pencarian (search)

Pengaruh TLB:

Jika terjadi TLB hit, semua struktur hanya membutuhkan 1 kali akses memori, sehingga perbedaan performa hampir tidak terasa.

3. Kasus Penggunaan

- Sistem 32-bit $\rightarrow$ Two-Level Page Table
- Sistem 64-bit $\rightarrow$ Three-Level / Multi-Level Page Table
- Inverted Page Table lebih efisien ketika:
 - Virtual address space sangat besar
 - Jumlah frame fisik terbatas
 - Banyak proses berjalan bersamaan

Tugas 5: Swapping Mechanism

Deskripsi

Simulasikan mekanisme swapping dengan berbagai strategi.

Yang Harus Dikerjakan

A. Basic Swapping Implementation

Buat program untuk mensimulasikan swap in/out:

```
typedef struct {  
    int pid;  
  
    int size;  
  
    int priority; int  
    in_memory; int  
    swap_count;  
  
    time_t last_access;  
  
} Process;  
  
void swap_out(Process *p); void
```

B. Swap Strategy Implementation Implementasikan 3 strategi victim selection:

- . FIFO: Swap out proses yang paling lama di memory
- . LRU: Swap out proses yang paling lama tidak diakses
- . Priority-based: Swap out proses dengan prioritas rendah

A. Simulation Scenario Test dengan skenario:

- Memory fisik: 1024 MB
- 10 proses dengan ukuran berbeda (50-200 MB) • Total kebutuhan memory: 1500 MB
- Simulasi akses random selama 100 time units

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_PROCESS 10
#define MEMORY_SIZE 1024
#define TIME_UNITS 100

typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int last_access;
    int load_time;
} Process;

Process p[MAX_PROCESS];
int used_memory = 0;
int current_time = 0;

/* Statistik */
int swap_in_count = 0;
int swap_out_count = 0;
int total_swap_time = 0;

/* ===== INITIALIZATION ===== */
void init_process() {
    for (int i = 0; i < MAX_PROCESS; i++) {
        p[i].pid = i;
        p[i].size = 50 + rand() % 151; // 50-200 MB
        p[i].priority = 1 + rand() % 10; // 1-10
        p[i].in_memory = 0;
        p[i].last_access = 0;
        p[i].load_time = 0;
    }
}

/* ===== VICTIM SELECTION ===== */
int select_fifo() {
    int idx = -1, oldest = 1e9;
    for (int i = 0; i < MAX_PROCESS; i++) {
        if (p[i].in_memory && p[i].load_time < oldest) {
            oldest = p[i].load_time;
            idx = i;
        }
    }
    return idx;
}

int select_lru() {
    int idx = -1, oldest = 1e9;
    for (int i = 0; i < MAX_PROCESS; i++) {
        if (p[i].in_memory && p[i].last_access < oldest) {
            oldest = p[i].last_access;
            idx = i;
        }
    }
}

}
return idx;
}

int select_priority() {
    int idx = -1, lowest = 1e9;
    for (int i = 0; i < MAX_PROCESS; i++) {
        if (p[i].in_memory && p[i].priority < lowest) {
            lowest = p[i].priority;
            idx = i;
        }
    }
    return idx;
}

/* ===== SWAP OPERATIONS ===== */
void swap_out(int idx) {
    p[idx].in_memory = 0;
    used_memory -= p[idx].size;
    swap_out_count++;
    total_swap_time += p[idx].size;
}

void swap_in(int idx) {
    while (used_memory + p[idx].size > MEMORY_SIZE) {
        int victim = select_lru();
        swap_out(victim);
    }
    p[idx].in_memory = 1;
    p[idx].load_time = current_time;
    p[idx].last_access = current_time;
    used_memory += p[idx].size;
    swap_in_count++;
}

/* ===== SIMULATION ===== */
void simulate(int strategy) {
    used_memory = 0;
    swap_in_count = 0;
    swap_out_count = 0;
    total_swap_time = 0;

    for (current_time = 0; current_time < TIME_UNITS; current_time++) {
        int access = rand() % MAX_PROCESS;

        if (!p[access].in_memory) {
            if (strategy == 0) swap_in(access);
            else if (strategy == 1) swap_in(access);
            else swap_in(access);
        }
        p[access].last_access = current_time;
    }
}

/* ===== MAIN ===== */
int main() {

```

```

    srand(time(NULL));
    init_process();

    printf("=== SWAPPING SIMULATION ===\n");
    printf("Memory: %d MB | Processes: %d | Time Units: %d\n",
           MEMORY_SIZE, MAX_PROCESS, TIME_UNITS);

    printf("Strategy | Swap-Out | Swap-In | Total Swap Time\n");
    printf("-----\n");

    simulate(0);
    printf("FIFO      | %8d | %7d | %15d\n",
           swap_out_count, swap_in_count, total_swap_time);

    simulate(1);
    printf("LRU       | %8d | %7d | %15d\n",
           swap_out_count, swap_in_count, total_swap_time);

    simulate(2);
    printf("Priority  | %8d | %7d | %15d\n",
           swap_out_count, swap_in_count, total_swap_time);

    return 0;
}

```

Outputnya:

```

sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano swapping_simulator.c
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ gcc swapping_simulator.c -o swapping_simulator
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ ./swapping_simulator
=== SWAPPING SIMULATION ===
Memory: 1024 MB | Processes: 10 | Time Units: 100

Strategy | Swap-Out | Swap-In | Total Swap Time
-----
FIFO      |      20 |      27 |          2644
LRU       |       0 |       3 |              0
Priority  |       0 |       0 |              0
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ |

```

Matriks performa

Strategy	Swap Out Count	Swap In Count	Total Swap Time	Avg Response Time
FIFO	20	27	2644	Paling lambat
LRU	0	3	0	Lebih cepat
Priority	0	0	0	Paling cepat

- Strategi yang meminimalkan thrashing:
LRU dan Priority-based, karena hampir tidak melakukan swap out sehingga proses jarang dipindahkan keluar masuk memori.
- Pengaruh ukuran memory terhadap performa:
Dengan memory fisik 1024 MB, sebagian besar proses masih dapat dipertahankan di memori. Jika ukuran memory diperkecil, jumlah swap out akan meningkat dan performa menurun.
- Kapan swapping lebih baik daripada killing process:
Swapping lebih baik digunakan ketika proses masih akan diakses kembali atau memiliki prioritas penting. Killing process lebih cocok untuk proses yang jarang digunakan atau tidak kritis.

Tugas 6: Architecture Analysis

Deskripsi

Analisis arsitektur memori pada sistem nyata.

Yang Harus Dikerjakan

A. Intel Architecture Analysis

Jalankan command berikut pada sistem Ubuntu:

```
# 1. Cek informasi CPU
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

# 2. Cek page size getconf
PAGE_SIZE

# 3. Cek TLB info cpuid
| grep -i tlb
```

Dokumentasikan dan analisis:

- Arsitektur prosesor (32-bit/64-bit)
- Page size yang digunakan
- Ukuran TLB
- Hierarchy cache (L1, L2, L3)

B. Memory Information

```
# 1. Total memory  
free -h
```

```
# 2. Virtual memory info vmstat  
1 5
```

```
# 3. Memory map dari proses cat  
/proc/self/maps
```

C. Analisis (Dalam Laporan)

. Bandingkan spesifikasi sistem Anda dengan:

- o Intel 32-bit (IA-32) architecture
- o Intel 64-bit (x86-64) architecture
- o ARMv8 architecture (jika tersedia)

. Jelaskan:

- o Berapa bit address space yang digunakan?

Bagaimana implementasi multi-level paging?

Berapa ukuran maksimum virtual memory?

```
#!/bin/bash

echo "=====
echo "❖ INTEL ARCHITECTURE ANALYSIS
echo "=====

echo "=== CPU ARCHITECTURE ==="
lscpu | grep -E "Architecture|CPU op-mode|Address sizes"
echo

echo "=== PAGE SIZE ==="
getconf PAGE_SIZE
echo "bytes"
echo

echo "=== TLB INFORMATION ==="
if command -v cpuid >/dev/null 2>&1; then
    cpuid | grep -i tlb | head -n 10
else
    echo "cpuid not installed"
    echo "Install with: sudo apt install cpuid"
fi
echo

echo "=== CACHE HIERARCHY ==="
lscpu | grep -E "L1d cache|L1i cache|L2 cache|L3 cache"
echo

echo "=== MEMORY INFORMATION ==="
cat /proc/meminfo | head -n 10
echo

echo "=== ADDRESS SPACE LIMITS ==="
ulimit -a | grep -i virtual
echo

echo "=== MEMORY MAP OF THIS PROCESS ==="
echo "PID: $$"
cat /proc/$$/maps | head -n 10
echo "... (truncated)"
echo

echo "=====
echo "❖ ANALYSIS COMPLETE
echo "=====
```

Outputnya:

```
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ nano intel_analysis.sh
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ chmod +x intel_analysis.sh
./intel_analysis.sh

=====
❖ INTEL ARCHITECTURE ANALYSIS
=====

=== CPU ARCHITECTURE ===
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                48 bits physical, 48 bits virtual

=== PAGE SIZE ===
4096
bytes

=== TLB INFORMATION ===
cpuid not installed
Install with: sudo apt install cpuid

=== CACHE HIERARCHY ===
L1d cache:                   128 KiB (4 instances)
L1i cache:                   128 KiB (4 instances)
L2 cache:                    2 MiB (4 instances)
L3 cache:                    4 MiB (1 instance)

=== MEMORY INFORMATION ===
MemTotal:                    3635428 kB
MemFree:                     3040272 kB
MemAvailable:                3175152 kB
Buffers:                     14752 kB
Cached:                      175720 kB
SwapCached:                  0 kB
Active:                      76000 kB
Inactive:                    247984 kB
Active(anon):                2648 kB
Inactive(anon):              134512 kB

=== ADDRESS SPACE LIMITS ===
virtual memory               (kbytes, -v) unlimited

=== MEMORY MAP OF THIS PROCESS ===
PID: 2031
5b4d5dc3e000-5b4d5dc6d000 r--p 00000000 00:30 1462 /usr/bin/bash
5b4d5dc6d000-5b4d5dd4c000 r-xp 0002f000 00:30 1462 /usr/bin/bash
5b4d5dd4c000-5b4d5dd86000 r--p 0010e000 00:30 1462 /usr/bin/bash
5b4d5dd86000-5b4d5dd8b000 r--p 00148000 00:30 1462 /usr/bin/bash
5b4d5dd8b000-5b4d5dd94000 rw-p 0014c000 00:30 1462 /usr/bin/bash
5b4d5dd94000-5b4d5dd9f000 rw-p 00000000 00:00 0
5b4d9cc6f000-5b4d9ccb1000 rw-p 00000000 00:00 0 [heap]
7123fec00000-7123fec28000 r--p 00000000 00:30 47592 /usr/lib/x86_64-linux-gnu/libc.so.6
7123fec28000-7123fecbd000 r-xp 00028000 00:30 47592 /usr/lib/x86_64-linux-gnu/libc.so.6
7123fecbd000-7123fee15000 r--p 001bd000 00:30 47592 /usr/lib/x86_64-linux-gnu/libc.so.6
... (truncated)

=====
❖ ANALYSIS COMPLETE
=====
sintiya@LAPTOP-PPED66E9:~/praktikum-memory/architecture$ |
```

Arsitektur	Address Space	Paging
IA-32 (32-bit)	32 bit	2-level paging
x86-64 (64-bit)	48–57 bit	4-level paging
ARMv8	48 bit	Multi-level paging

- Arsitektur prosesor: 64-bit (x86-64)
- Page size: 4096 bytes (4 KB)
- Cache hierarchy:
 - L1 (instruction & data)
 - L2 (per core)
 - L3 (shared)
- TLB: Digunakan untuk mempercepat translasi alamat virtual ke fisik

Penjelasan Teknis

- Bit address space:
Sistem x86-64 menggunakan hingga 48 bit virtual address.
- Multi-level paging:
Menggunakan 4 level page table (PML4 → PDPT → PD → PT) untuk menghemat memori.
- Maksimum virtual memory:
Secara teoritis hingga 256 TB pada arsitektur x86-64.